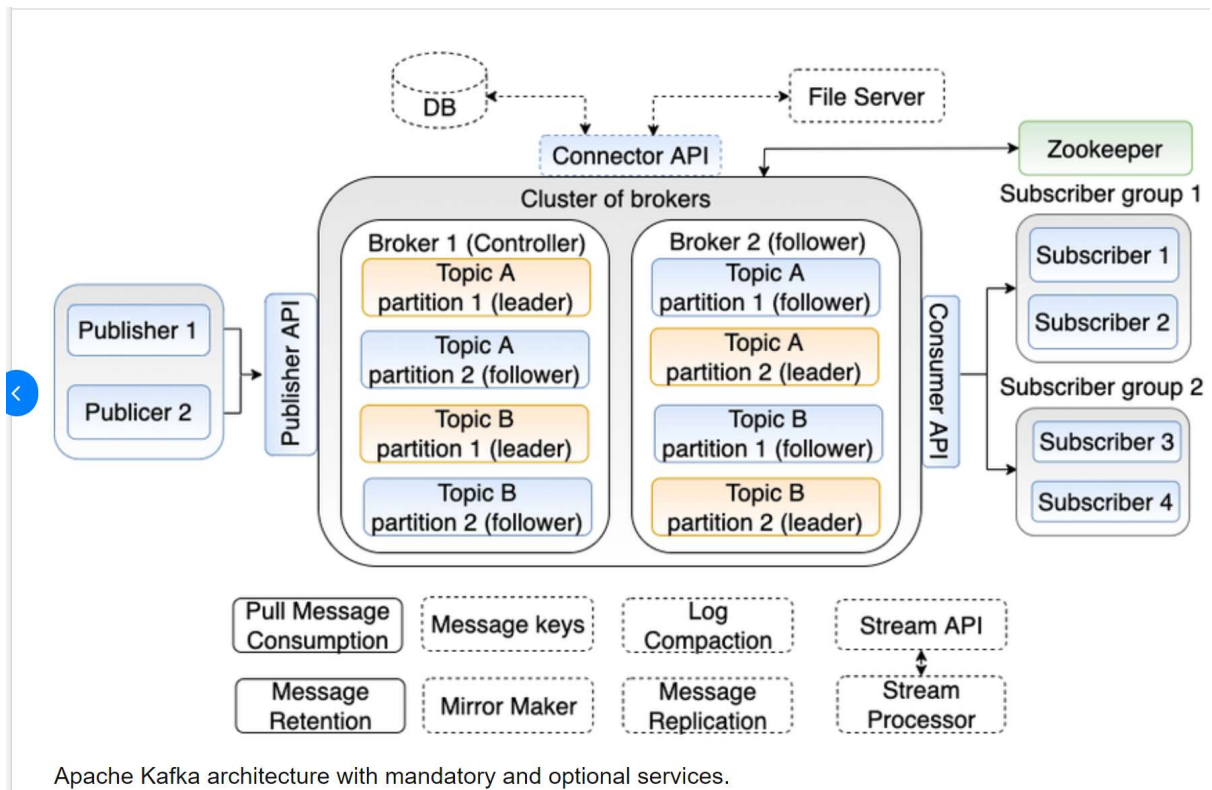


Apache Kafka with Java

1. Apache Kafka Overview

Theory

- Apache Kafka is an open-source distributed streaming platform used for handling real-time data streaming.
- Kafka is known for its scalability, fault-tolerance, and ability to handle high volumes of data.



Key Concepts:

1. **Topics:** Topics are categories or feeds to which messages are published in Kafka.
2. **Partitions:** Topics are divided into partitions, which allow for parallel processing and scalability.
3. **Messages:** Messages are units of data in Kafka, consisting of a key, a value, and additional metadata.

4. **Producers:** Producers publish messages to Kafka topics.
5. **Consumers:** Consumers subscribe to topics and consume messages from Kafka.
6. **Consumer Groups:** Consumers can be organized into consumer groups, where each message is consumed by only one consumer within a group.
7. **Brokers:** Brokers are the servers that handle storage and replication of Kafka topic partitions.
8. **ZooKeeper:** ZooKeeper is used for managing and coordinating Kafka brokers and consumers.

Publish-Subscribe Model:

- Kafka follows a publish-subscribe model, where producers publish messages to topics, and consumers subscribe to topics to consume messages.
- Topics can have multiple producers and multiple consumers, forming a distributed and scalable architecture.

Key Features:

1. **Scalability:** Kafka can scale horizontally across multiple machines to handle large volumes of data.
2. **Fault-tolerance:** Kafka provides replication of data across multiple brokers, ensuring high availability and durability.
3. **High-throughput:** Kafka is optimized for high write and read throughput, making it suitable for real-time streaming applications.
4. **Message Retention:** Kafka retains messages for a configurable period, allowing consumers to replay messages and perform batch processing.
5. **Exactly-once Semantics:** Kafka supports exactly-once message processing semantics, ensuring no data loss or duplication.
6. **Extensibility:** Kafka integrates with other systems through Kafka Connect API and allows stream processing through Kafka Streams API.

Use Cases:

1. **Real-time Streaming:** Kafka is widely used for real-time data ingestion, processing, and analytics in domains like finance, e-commerce, and social media.

2. **Log Aggregation:** Kafka can collect logs from multiple sources and store them in a central location, facilitating analysis and monitoring.
3. **Event Sourcing:** Kafka serves as an event sourcing platform, capturing and storing all events in a system for easy state reconstruction and auditing.
4. **Data Integration:** Kafka acts as a data backbone for integrating and synchronizing data across different systems and applications.

Kafka Ecosystem:

1. **Kafka Connect:** Kafka Connect is a framework for building connectors that import/export data between Kafka and external systems.
2. **Kafka Streams:** Kafka Streams is a client library for building real-time stream processing applications.
3. **KSQL:** KSQL is a streaming SQL engine for querying and transforming Kafka topics using SQL-like syntax.
4. **Schema Registry:** The Schema Registry stores and manages Avro schemas for Kafka messages, ensuring compatibility and data consistency.

Apache Kafka is an open-source distributed event streaming platform used for real-time data streaming, messaging, log aggregation, event sourcing, and event-driven architectures.

Main capabilities:

- High throughput
- Fault tolerance
- Horizontal scalability
- Durability
- Low latency

2. Key Components

- **Topics:** Logical categories where records are published.
- **Partitions:** Divide topics for parallel processing.
- **Records:** Each record contains an optional key, value, timestamp, and metadata.
- **Producers:** Publish records.
- **Consumers:** Read records.
- **Consumer Groups:** One consumer per partition within the same group.
- **Brokers:** Store partitions and replicas.
- **KRaft:** Modern metadata management replacing ZooKeeper.

3. Important Correction: ZooKeeper vs KRaft

The original notes mention ZooKeeper as required. Modern Kafka (4.x) removes ZooKeeper completely and uses KRaft. Kafka 3.x introduced KRaft as the replacement.

4. Kafka Architecture

Flow: Producer → Topic → Partition Leader → Replica Followers → Consumer → Offset Commit.

5. Kafka Features

Feature	Purpose
Scalability	Add brokers horizontally
Fault Tolerance	Replicated partitions
High Throughput	Optimized reads and writes
Message Retention	Replay historical events
Exactly Once	Transactions + idempotence
Kafka Connect	External integrations
Kafka Streams	Java stream processing

6. Modern Installation (KRaft)

```
bin/kafka-storage.sh random-uuid
bin/kafka-storage.sh format -t <cluster-id> -c config/kraft/server.properties
bin/kafka-server-start.sh config/kraft/server.properties
```

7. Kafka CLI Commands

Create topic:

```
bin/kafka-topics.sh --create --topic my-topic --bootstrap-server localhost:9092
--partitions 3 --replication-factor 1
```

List topics:

```
bin/kafka-topics.sh --list --bootstrap-server localhost:9092
```

Describe topic:

```
bin/kafka-topics.sh --describe --topic my-topic --bootstrap-server
localhost:9092
```

Console producer:

```
bin/kafka-console-producer.sh --topic my-topic --bootstrap-server
localhost:9092
```

Console consumer:

```
bin/kafka-console-consumer.sh --topic my-topic --bootstrap-server
localhost:9092 --from-beginning
```

8. Java Project Setup

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-clients</artifactId>
  <version>4.1.0</version>
</dependency>
```

9. Java Producer

Configuration:

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSeriali
zer");
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSeria
lizer");
KafkaProducer<String,String> producer = new KafkaProducer<>(props);
```

Send with callback:

```
ProducerRecord<String,String> record =
    new ProducerRecord<>("my-topic", "key1", "Hello Kafka");
producer.send(record, (metadata, exception) -> {
    if(exception==null) {
        System.out.println(metadata.partition());
        System.out.println(metadata.offset());
    } else {
        exception.printStackTrace();
    }
});
producer.flush();
producer.close();
```

10. Java Consumer

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("group.id", "my-group");
props.put("key.deserializer", "org.apache.kafka.common.serialization.StringDeser
ializer");
props.put("value.deserializer", "org.apache.kafka.common.serialization.StringDes
erializer");
props.put("auto.offset.reset", "earliest");
props.put("enable.auto.commit", "false");
KafkaConsumer<String,String> consumer = new KafkaConsumer<>(props);
consumer.subscribe(List.of("my-topic"));
```

Polling loop:

```
while(true) {
    ConsumerRecords<String,String> records =
        consumer.poll(Duration.ofSeconds(1));
    for(ConsumerRecord<String,String> record: records) {
        System.out.println(record.value());
    }
}
```

```

        consumer.commitSync();
    }
}

```

11. Serialization

Kafka stores bytes. Common Java serializers:

- StringSerializer
- IntegerSerializer
- LongSerializer
- Jackson JSON serializer
- Avro
- Protobuf

12. Consumer Groups and Offsets

If a topic has three partitions and three consumers in the same group, each consumer gets one partition. Extra consumers remain idle.

Strategy	Behavior
Auto Commit	Simple but may lose progress
Manual Commit	More reliable
Commit Before Processing	At-most-once
Commit After Processing	At-least-once

13. Producer Configurations

Property	Purpose
acks=all	Highest durability
enable.idempotence=true	Prevent duplicates
retries	Retry failures
linger.ms	Batch delay
batch.size	Larger batches
compression.type=zstd	Better compression
max.in.flight.requests.per.connection	Parallel requests

14. Consumer Configurations

Property	Purpose
group.id	Consumer group
auto.offset.reset	earliest/latest
enable.auto.commit	Automatic commits
max.poll.records	Batch size
session.timeout.ms	Failure detection
heartbeat.interval.ms	Heartbeats

15. Best Practices

Producer:

- acks=all

- `enable.idempotence=true`
- `compression.type=zstd`
- `linger.ms=5`
- `batch.size=32768`

Consumer:

- `enable.auto.commit=false`
- `commitSync()` after processing
- Tune `max.poll.records`
- Tune heartbeat settings

16. Kafka Connect and Kafka Streams

Kafka Connect integrates Kafka with external systems like MySQL, PostgreSQL, Elasticsearch, MongoDB, and S3.

Kafka Streams is a Java library for stream processing.

```
StreamsBuilder builder = new StreamsBuilder();
builder.stream("orders")
    .filter((k,v)->v.contains("paid"))
    .to("paid-orders");
```

17. Java Interview Questions

Q: What is Kafka?

A: Distributed event streaming platform.

Q: Topic vs Partition?

A: Topic is logical; partition is physical.

Q: What is a Consumer Group?

A: Consumers share partitions for load balancing.

Q: Why use a key?

A: Same key goes to the same partition.

Q: Ordering guarantee?

A: Only within one partition.

Q: `commitSync` vs `commitAsync`?

A: Sync is blocking; async is faster.

Q: What is idempotence?

A: Prevents duplicates during retries.

Q: How does Kafka provide fault tolerance?

A: Leader-follower replication.

Q: ZooKeeper vs KRaft?

A: KRaft is the modern replacement.

18. Corrections Applied to Original Notes

- ZooKeeper replaced with KRaft.
- Added `enable.idempotence=true`.
- Added `zstd` compression.
- Manual `commitSync()` shown.
- Modern installation commands.